

Processor Prototyping Lab

Final Report

Trent Seaman & Xinyu Liu

GTA: Christopher Miotto

Due Date: 5/3/2026

1 Executive Overview

This report presents the design and performance evaluation of a RISC-V processor developed across four major versions: a single-cycle baseline, a 5-stage pipelined processor without caches, a pipelined processor with a cache hierarchy, and a dual-core multicore processor with MSI cache coherence and LR-SC support. Each design was implemented in SystemVerilog and synthesized through the standard mapping flow. The pipelined core includes a forwarding unit, a hazard unit, and a 2-bit saturating-counter branch predictor. The cache hierarchy consists of a direct-mapped instruction cache and a 2-way set-associative data cache with writeback policy. The multicore builds on this with a blocking bus controller that arbitrates between two cores, implements the MSI coherence protocol, including cache-to-cache transfers and coherence writebacks, and LR-SC Zalrsc subset of A-extension for atomic support.

The Results section compares all four designs using the merge-sort benchmark (dual-mergesort for the multicore version) at memory latency of 6, reporting synthesis frequency, average CPI, total execution time, ideal single-instruction latency, FPGA resource utilization, and the speedup from the single-threaded to dual-threaded mergesort execution. A second comparison sweeps the memory latency across 0, 2, 6, and 10 to identify the point. The pipelined design is expected to outperform the single-cycle baseline by increasing the frequency. Adding on cache hierarchy with pipelined design is expected to dominate at higher memory latencies by decreasing average memory access time. The multicore design is expected to deliver a measurable speedup on the dual-threaded workload over its single-threaded execution, demonstrating the benefit of parallel execution.

2 Design

This section presents RTL diagrams for the pipelined processor with LR/SC, the cache hierarchy, and the multicore coherence control.

2.1 Pipelined Processor

This is a 5-stage pipelined RISC-V core (IF, ID, EXE, MEM, WB) shown in Figure 1. A forwarding unit, hazard unit, 2-bit saturating-counter branch predictor, and LR/SC support in the datapath are fully implemented.

2.2 Multicore Processor

The multicore design instantiates two cores, each with its own datapath, instruction cache, and data cache, sharing a single bus controller and RAM (Figure 2). The MSI coherence protocol and LR/SC (the Zalrsc subset of the A-extension) are implemented.

2.3 Cache Hierarchy and Data Cache Controller

The instruction cache (Figure 4) is direct-mapped with 16 entries and a 1-word block size, and is read-only. An instruction hit is resolved in one cycle; on a miss, the processor is stalled until the instruction is read from RAM.

The data cache is 2-way set-associative with 8 sets and 2-word blocks (128 B total), as shown in Figure 5. The control flow of the dcache controller is divided into two parts: Operation and Coherence.

The Operation part of the dcache controller handles dirty block eviction and memory fetching for two data blocks. When the processor halts, it flushes all dirty blocks back to RAM. MSI state transitions triggered by processor accesses are implemented in this part. Atomic access is implemented with a reservation register: during an LR, the register records the address and a BusRdX is issued; during an SC, the reservation is checked, the other cache is invalidated, and once the store completes, the local reservation register is also invalidated.

The reservation register is additionally invalidated on any local dcache write between the LR and SC. The M state is not updated with BusRdX data during an LR.

The Coherence part of the dcache controller handles snooping the other core, along with the MSI state transitions and responses triggered by snooped bus operations. MSI responses include supplying a data block over the bus and writing back the snooped address on a snoop hit.

2.4 Bus Controller

The bus controller is an FSM containing 14 states, shown in Figure 7. It is a blocking bus controller. Instruction grants are issued directly from the IDLE state, while data grants are issued from the ARBIT state. During a normal data writeback, the WB1 and WB2 states are granted. During a data read, the bus controller first snoops the other cache to check whether it contains the requested data, and then either reads from RAM or receives the block supplied by the other dcache. A coherence writeback occurs after the WAIT state, once the other cache has determined whether the writeback is required.

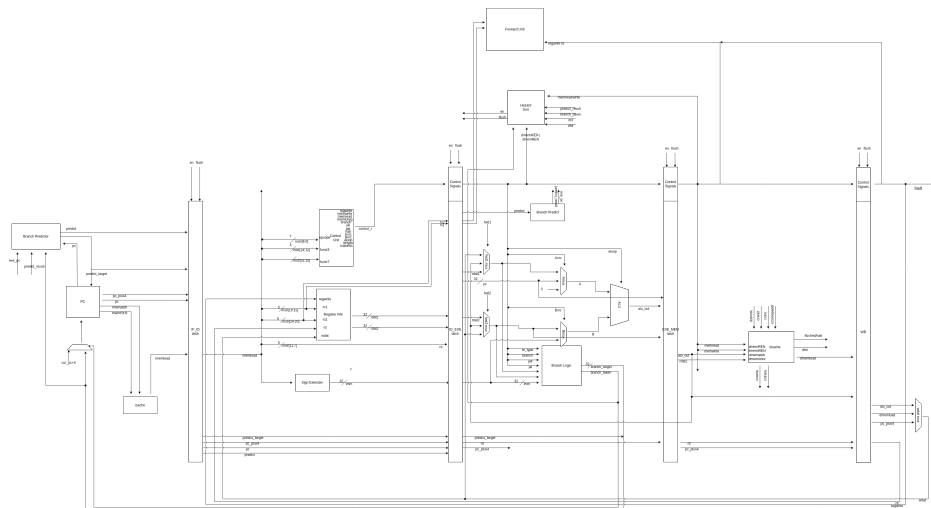


Figure 1: Pipelined processor datapath.

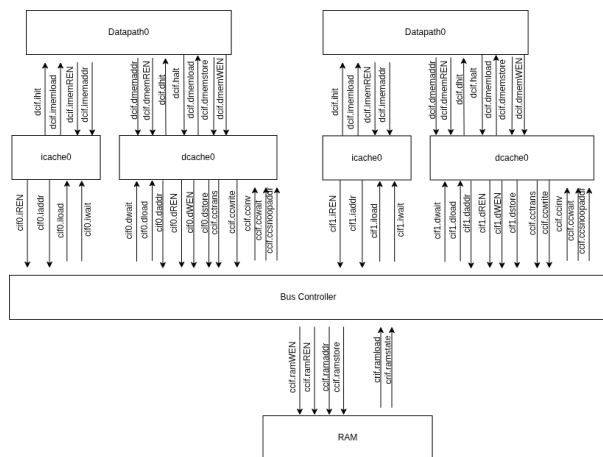


Figure 2: Dual-core top-level.

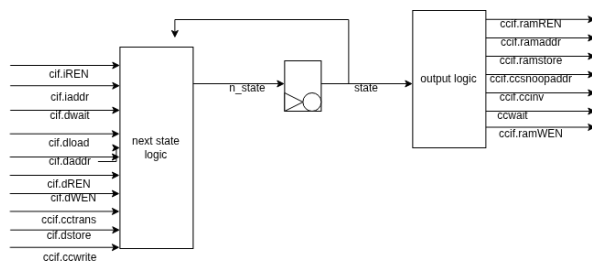


Figure 3: Bus controller FSM block diagram.

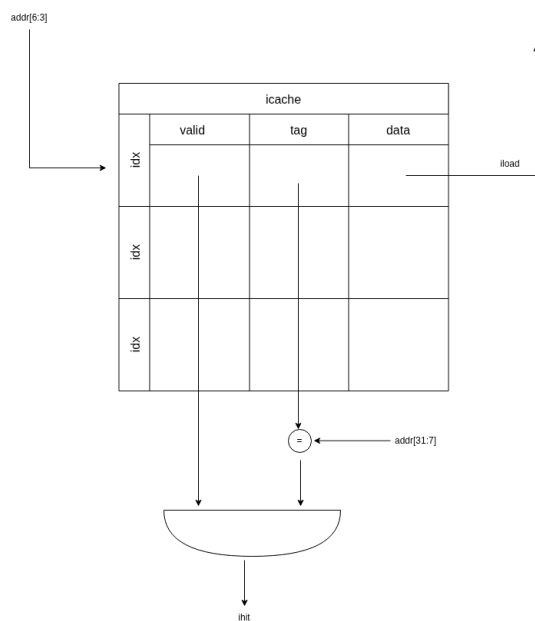


Figure 4: Instruction cache.

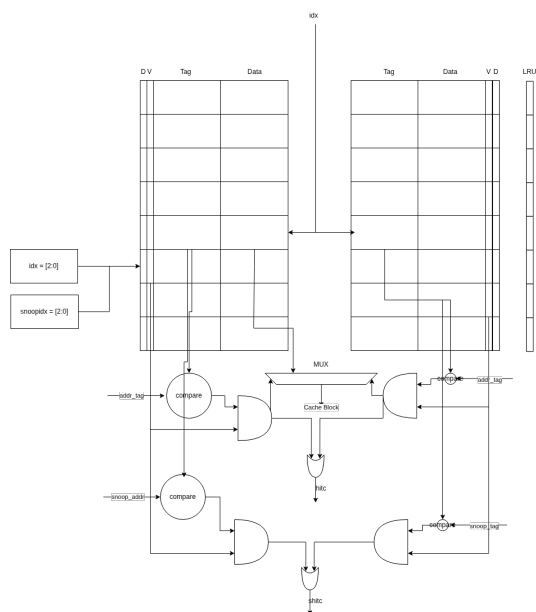


Figure 5: Data cache structure.

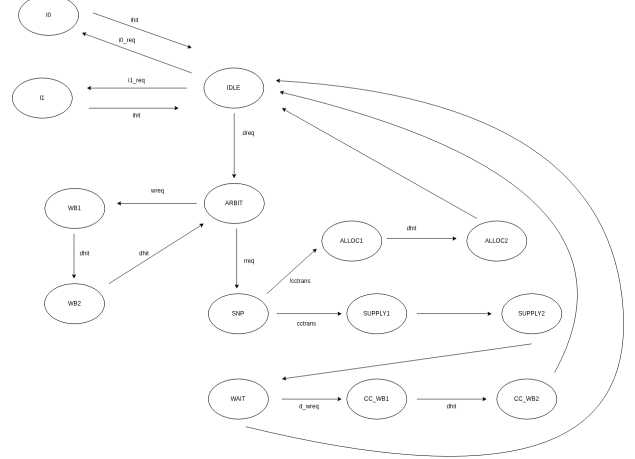
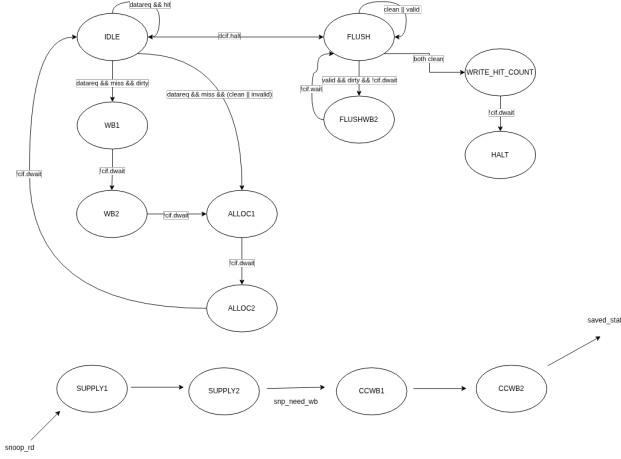


Figure 6: Data cache controller state diagram. Figure 7: Bus controller state diagram.

3 Results

The performance of each processor design was evaluated using the merge-sort benchmark. The primary metrics for comparison at a fixed memory latency of 6 are summarized in Table 1.

The formulas used to compute each performance metric are summarized below.

- **Frequency:** $f = \min\left(\frac{f_{\text{MAIN}}}{2}, f_{\text{CPUCLK}}\right)$
- **Average CPI:** $\text{CPI} = \frac{\text{Total Cycles}}{\text{IC}}$, where IC means instruction count.
- **Ideal Latency:** $T_{\text{ideal}} = \frac{N_{\text{stages}}}{f}$, where $N_{\text{stages}} = 1$ for the single-cycle design and $N_{\text{stages}} = 5$ for the pipelined designs.
- **Total Execution Time:** $T_{\text{exec}} = \text{Total Cycles} \cdot \frac{1}{f} = \text{IC} \cdot \text{CPI} \cdot \frac{1}{f}$

The speedup from the single-threaded multicore execution to the dual-threaded execution at a memory latency of 6 was calculated as $S = T_{\text{seq}}/T_{\text{par}} = 1.08x$.

Metric	Single	Pipe	Cache	MC (1-T)	MC (2-T)
Freq (MHz)	73.90	76.63	66.81	71.89	59.70
Avg CPI	6.85	5.49	1.56	1.56	2.87
Exec Time (ns)	501,326	387,589	126,358	117,430	108,559
Ideal Lat (ns)	13.53	65.25	74.84	69.55	83.75
LUTs	1,883	2,563	4,016	10,130	10,214
Registers	1,198	2,604	5,154	10,498	10,490
F7/F8 Muxes	320	473	967	2,113	2,112

Table 1: Processor Performance and Resource Utilization (Memory Latency = 6)

3.1 Memory Latency Sweep

To determine the efficiency of the cache hierarchy, the total execution time was captured across various memory latencies (Table 2). The data is collected by running mergesort algorithm.

Design Version	Lat 0	Lat 2	Lat 6	Lat 10
Single-Cycle	124,640	250,663	501,326	751,989
Pipeline (No Cache)	94,409	204,319	387,589	570,859
Pipeline (With Cache)	93,798	111,241	126,358	141,476
Multicore (1-Thread)	100,450	103,380	117,430	131,479
Multicore (2-Thread)	80,440	89,598	108,559	127,621

Table 2: Total Execution Time (ns) vs. Memory Latency

4 Conclusion

The evaluation demonstrates that while the single-cycle processor provides a simple baseline, the 5-stage pipeline improves performance at low latencies. However, as memory latency increases, the pipeline without caches suffers from extreme stalling, with execution time

rising from 94,409 ns at latency 0 to 570,859 ns at latency 10. The addition of the cache hierarchy successfully goes against this, maintaining a better performance curve. Based on the data sweep, the cache hierarchy begins to provide a performance advantage even at a RAM latency of 0, where it already outperforms the non-cached pipeline by approximately 611 ns.

The multicore implementation achieve a speedup of 1.08x on the dual-threaded merge-sort workload at a memory latency of 6. While the increased hardware complexity results in a lower synthesis frequency (59.70 MHz) and a higher CPI due to bus arbitration overhead, the parallel execution allows the system to process the sorting algorithm more efficiently than any single-core configuration. The significant increase in FPGA resource utilization reaching over 10,000 LUTs and registers shows the trade-off between hardware area and parallel throughput.

5 Contributions

We collaborated throughout every part of the design.

- **Xinyu Liu:** Icache, Bus Controller, LR-SC Hardware, Dcache TB, Dcache MSI Protocol TB
- **Trent Seaman:** Dcache, Dcache MSI Protocol, Parallel Algorithm, Icache TB, Bus Controller TB